

Title: Supporting `offsetof` for Stable-layout Classes
Document number: P0545R0
Date: 2016-10-23
Project: ISO JTC1/SC22/WG21: Programming Language C++
Audience: LEWG, EWG
Reply-to: Andrey Semashev <andrey.semashev at gmail dot com>

1. Introduction

This proposal introduces a *stable layout class* definition and extends the set of types `offsetof` is required to support.

2. Motivation and Scope

Currently (according to [N4618](#), [support.types.layout]/1), `offsetof` is required to work for only standard-layout classes. This definition prohibits portable code from using `offsetof` in other useful contexts. For instance, consider the following classes:

```
struct A
{
    int a;
};

struct B : A
{
    int b;
};
```

Imagine that an application needs to pass objects of type `B` between multiple processes via shared memory. The classes are trivially copyable, so the simplest way to do that is to use `std::memcpy`. Occasionally, a process needs only one member of `B` (say, `B::b`). In order to extract it from the shared memory storage occupied by the complete object of `B`, the application needs to know the offset of `B::b` relative to the starting address of the `B` object. This offset is normally provided by `offsetof`, but formally the macro cannot be applied in this case because `B` is not a standard-layout class. That is because more than one class in the hierarchy has non-static data members.

It is understandable that the current definition of `offsetof` in C++ mostly just refers to the C standard, for the best compatibility with C. However, C++ offers more advanced means for type definition, including type inheritance, methods, constructors and destructors, yet it doesn't extend `offsetof` to support these types even when feasible. This proposal attempts to mitigate this omission.

2.1. Alternative Solutions

A number of alternative solutions could be used to achieve the desired functionality without extending `offsetof`. This section offers a discussion of these alternatives and the reasoning for still preferring `offsetof` extension.

2.1.1. Convert structures to standard layout

One possible alternative is to convert `B` from the above example to a standard-layout class, replacing inheritance with encapsulation like this:

```
struct standard_layout_B
{
    A a;
    int b;

    standard_layout_B() = default;
    standard_layout_B(A const& a_) : a(a_) {}
    operator A& () { return a; }
    operator A const& () const { return a; }
};
```

While this approach solves the problem of formal incompatibility with `offsetof`, it adds problems of its own:

1. It contradicts the intent of the developer, if `B` was supposed to be a natural extension of `A` (i.e. `B` should have been derived

from A in every other case, if not for `offsetof`). As a result, it obscures the program design, making it more difficult to understand and maintain.

2. The `standard_layout_B` class is not equivalent to B on a semantic level. For instance, since `standard_layout_B` does not inherit from A, pointers to `standard_layout_B` cannot be converted to pointers to A (`[conv.ptr]/3`); pointers to members of A cannot be converted to pointers to members of `standard_layout_B` (`[conv.mem]/2`). Standard type traits, such as `is_base_of` will also report `standard_layout_B` and A as unrelated. This limits interchangeability between A and `standard_layout_B` in the surrounding code.
3. The `standard_layout_B` class is not equivalent to B on a syntactic level. Members of A are not members of `standard_layout_B`, and every mention of an A member will have to involve either mentioning `standard_layout_B::a` or obtaining a reference to it (e.g. by calling the type conversion operator defined in `standard_layout_B`). This greatly complicates writing generic code that is supposed to work with A, B and other classes that derive from A, similar to B. With multiple such Bs the approach quickly shows poor scalability.

To illustrate these problems, let's consider the following class hierarchy, which is a simplified version of a real code. Imagine a media processing application that consists of multiple processes exchanging video and audio frames via shared memory. Each frame is associated with a set of metadata expressed by one of these structures:

```
struct frame { long timestamp; };

struct audio_frame : frame { ... };
struct video_frame : frame { ... };

struct raw_audio_frame : audio_frame { ... };
struct encoded_audio_frame : audio_frame { ... };

struct raw_video_frame : video_frame { ... };
struct encoded_video_frame : video_frame { ... };
```

Note that the hierarchy may be deeper and may include, for example, classes that correspond to particular video and audio codecs. Rewriting them as standard-layout classes would result in this:

```
struct frame { long timestamp; };

struct audio_frame { frame f; ... };
struct video_frame { frame f; ... };

struct raw_audio_frame { audio_frame af; ... };
struct encoded_audio_frame { audio_frame af; ... };

struct raw_video_frame { video_frame vf; ... };
struct encoded_video_frame { video_frame vf; ... };
```

Now, in order to reference `frame::timestamp` the user's code has to be different, depending on the actual type of the class the code is dealing with:

```
frame f;
audio_frame af;
video_frame vf;
raw_audio_frame raf;
encoded_audio_frame eaf;
raw_video_frame rvf;
encoded_video_frame evf;

f.timestamp = 10;
af.f.timestamp = 10;
vf.f.timestamp = 10;
raf.af.f.timestamp = 10;
eaf.af.f.timestamp = 10;
rvf.vf.f.timestamp = 10;
evf.vf.f.timestamp = 10;
```

A possible workaround for this is to introduce type traits or accessors to all members of the structures (including data members and functions). For example:

```
struct frame
{
    long m_timestamp;

    long& timestamp() noexcept { return m_timestamp; }
```

```

    long const& timestamp() const noexcept { return m_timestamp; }
};

struct audio_frame
{
    frame m_f;
    ...

    long& timestamp() noexcept { return m_f.timestamp(); }
    long const& timestamp() const noexcept { return m_f.timestamp(); }
    ...
};

struct video_frame
{
    frame m_f;
    ...

    long& timestamp() noexcept { return m_f.timestamp(); }
    long const& timestamp() const noexcept { return m_f.timestamp(); }
    ...
};

struct raw_audio_frame
{
    audio_frame m_af;
    ...

    long& timestamp() noexcept { return m_af.timestamp(); }
    long const& timestamp() const noexcept { return m_af.timestamp(); }
    ...
};

struct encoded_audio_frame
{
    audio_frame m_af;
    ...

    long& timestamp() noexcept { return m_af.timestamp(); }
    long const& timestamp() const noexcept { return m_af.timestamp(); }
    ...
};

struct raw_video_frame
{
    video_frame m_vf;
    ...

    long& timestamp() noexcept { return m_vf.timestamp(); }
    long const& timestamp() const noexcept { return m_vf.timestamp(); }
    ...
};

struct encoded_video_frame
{
    video_frame m_vf;
    ...

    long& timestamp() noexcept { return m_vf.timestamp(); }
    long const& timestamp() const noexcept { return m_vf.timestamp(); }
    ...
};

```

While that unifies access to all fields of the structures, this also adds a lot of verbosity to the structures' definition. Adding, removing, or modifying any member of the base structures also requires similar modifications to all containing structures. This is much more tedious and error prone than the original code that used inheritance. The author is also convinced that the original code is a more appropriate design choice from the standpoint of semantic relations between the classes, as each derived class is a specialization and extension of its base classes.

2.1.1.1. Aren't standard-layout classes required for inter-process communication anyway?

The short answer to that question is no, they are not required. As long as the programs exchanging data are built with compilers that implement the same ABI specification, or different ABI specifications that are compatible with regard to types' representation, these programs shall be compatible. This guarantee is given by the ABI specification, not by the C++ standard. The standard-layout property by itself does not guarantee binary compatibility, nor is it a required precondition for one. What

standard-layout offers is a set of restrictions on C++ types so that they are compatible, on the language level, with a similar structure in another language (primarily, the C language), provided that ABI specifications for the two languages are compatible. However, when compatibility with languages other than C++ is not required, the restrictions imposed by a standard-layout class can be too limiting. Even when the two processes comply with different ABIs that are not fully compatible, there can be a less restricted subset of C++ features that can be used portably.

The existing practice on most current platforms is that there is one C++ ABI specification per target architecture that is supported by most or all compilers on the platform. One notable exception is Windows, where there are two commonly used ABIs: Microsoft's (supported by MSVC, Intel compiler, clang-cl) and GCC (supported by MinGW, MinGW-w64, clang). While different, these ABIs are still very compatible with regard to types' representation: all fundamental types, enums, and simple structures like `A`, `B` and `C` defined in this proposal, are compatible. On the other hand, even a structure as simple as `A` could not have been used for data exchange if the two ABIs defined their `int` representation differently.

It should also be noted that the programs that use C++ structures directly for data exchange are often parts of the same framework and are compiled by the same compiler, thus eliminating any potential ABI incompatibility. When data exchange is supposed to be carried out with external parties, it is normal to expect a more formal description of the exchange protocol that does not include any C/C++ structures.

2.1.2. Use serialization for data exchange

Another alternative solution to the original problem would be to implement (de)serialization of `A` and `B` to a binary format understood by all processes involved in data exchange. Such a format could potentially allow a fast extraction of a single data member without having to decode the full structure, thus eliminating the need for `offsetof` in the first place. This approach is often used in other types of data exchange, such as network-based or file-based data exchange.

The downside of serialization is that it entails a certain cost, on both development and runtime performance. Let's consider the following, probably the simplest implementation of serialization for `B`:

```
struct serialized_B
{
    int A_a; // corresponds to A::a
    int B_b; // corresponds to B::b
};

void serialize(B const& b, unsigned char* p)
{
    serialized_B s;
    s.A_a = b.a;
    s.B_b = b.b;

    std::memcpy(p, &s, sizeof(s));
}

void deserialize(const unsigned char* p, B& b)
{
    serialized_B s;
    std::memcpy(&s, p, sizeof(s));

    b.a = s.A_a;
    b.b = s.B_b;
}
```

The above code needs to be written for every `B` (and probably `A` to maintain the same API for storing and loading the structures to/from shared memory) and kept in sync with the structures as the code evolves. There is also a runtime overhead if the compiler is not able to optimize away the extra memory copy in `serialize()/deserialize()` (e.g. if `serialized_B` has different binary representation than `B`).

This imposed cost is likely justified if binary format of messages is important, like when the messages are transmitted to another machine and can be received by a foreign receiver. That is not the case when the messages are exchanged locally, between the processes that are already binary compatible; the cost can be avoided in this case.

3. Impact on the Standard

This proposal is a pure extension to the C++ language and standard library. Its intent is to define behavior that was previously not defined, in a way that most current implementations already work. No existing valid code is made invalid or changes its behavior.

4. Design Decisions

4.1. Trivially-copyable Types

With the current standard, it should be possible to extend `offsetof` to support at least trivially-copyable types. Given the `A` and `B` classes from the earlier example, the following code is expected to work:

```
B b1, b2;
int* p = &b2.b;

b1.a = 1;
b1.b = 2;

std::memcpy(&b2, &b1, sizeof(B));

assert(*p == 2);
```

This follows from [basic.types]/3. In particular, this also means that the relative position of `B::b` within `B` is constant and does not depend on the particular object of `B`. This is enough for `offsetof` to be able to operate on this type as expected.

4.2. Non-trivially-copyable Classes

It is the author's opinion that trivial copyability, or copyability at all, is not the property that defines binary layout of the type. For example, let's slightly modify the previously given `B` class as follows:

```
class C : public A
{
public:
    int c;
    char str[1024];

    C() : c(10)
    {
        str[0] = '\0';
    }

    C(C const& that) : c(that.c)
    {
        std::strcpy(str, that.str);
    }

    C& operator= (C const& that)
    {
        c = that.c;
        std::strcpy(str, that.str);
        return *this;
    }
};
```

The type `c` is no longer trivially-copyable, but is it no longer viable for inter-process communication? Is the mere presence of a user-defined copy constructor and assignment operator the limiting factor that prevents this? The author believes not, and indeed in the most widespread ABIs ([Itanium C++ ABI](#), as well as Microsoft Windows x64 ABI) binary layout does not depend on the presence of the special member functions. The author believes that there is no practical benefit in leaving the opportunity to do otherwise. This proposal aims to add support for `offsetof` for such types.

4.3. Classes with Virtual Functions

The most common implementation of virtual functions involves a virtual function table (vtable), a pointer to which is stored as a hidden data member in class objects. The size and position of this hidden member is constant, so in most cases virtual functions do not preclude `offsetof` from working as expected. In fact, some implementations (GCC 6.2, Clang 3.8, MSVC 19) do support `offsetof` with classes having virtual functions.

However, it is possible that other implementations exist. For instance, an implementation could use another hidden field that would contain an offset to data members of the final class object. Also, classes with virtual functions are not applicable for interprocess communication as described in the motivating example. For these reasons this proposal does not require support for classes with virtual functions and leaves them conditionally-supported. A future proposal could require such support if the

need appears.

4.4. Classes with Virtual Base Classes

Like virtual functions, virtual base classes also typically introduce a hidden data member (`vtordisp`). However, unlike virtual functions, this member is used to calculate the address of the virtual base class subobject at runtime. It is possible to perform that calculation at compile time when the final type of the complete object is known (for instance, if the final object is visible in the context of that calculation, or the class is marked as `final`), but in general the fact that runtime resolution may be required rules out universal support for such classes in `offsetof`. This proposal leaves such classes conditionally-supported by `offsetof`.

5. Impact on Existing Implementations

The currently widespread compilers (GCC 6.2, Clang 3.8, MSVC 19) all support `offsetof` for classes A, B and C described above, although some of the compilers emit warnings, as currently required by N4618. The implementation complexity is expected to be minimal.

6. Technical Specifications

6.1. Stable-layout Class Definition

Since the standard does not currently define a suitable category of types, this proposal introduces the definition of a *stable-layout class*, which is a class that satisfies the following conditions:

- no virtual base classes
- no virtual functions
- no non-static data members of types other than one of the following: a (possibly cv-qualified) scalar type or a stable-layout class, an array of such a type, or a reference type
- no non-stable-layout base classes

Objects of a stable-layout class shall be guaranteed to occupy contiguous bytes of storage. For a stable-layout class, it shall be guaranteed that relative positions (offsets) of non-static data members are known at compile time and constant across all objects of that class. These offsets shall account for any possible padding that is added between non-static data members to achieve alignment.

Note that this definition includes trivially copyable and standard-layout classes. Scalar types, stable-layout class types, arrays thereof and cv-qualified versions of those types shall be collectively called *stable-layout types*.

To allow for testing if a type is a stable-layout type, this proposal also adds a new type trait `is_stable_layout`.

6.2. Additional Restrictions on Member Designator

The definition of `offsetof(type, member-designator)` in [support.types.layout]/1 refers to the C standard, and it only requires `&(t.member-designator)` to evaluate to an address constant (here, `t` is an object of class type). Since this proposal allows using `offsetof` with non-trivial types, `member-designator` can now identify a reference member. Taking the address of a reference member would result in the address of a referred object, which is not the intended effect of `offsetof`. For this reason, this proposal prohibits the use of references in a `member-designator`; the behavior is undefined if this requirement is violated. Note that this rule applies if references are used at any level of `member-designator`, if it identifies a nested member. For example:

```
struct Bad
{
    int& n;
    A& a;
    int x;
};

offsetof(Bad, n);    // undefined behavior, Bad::n is a reference
offsetof(Bad, a.a);  // undefined behavior, Bad::a is a reference
offsetof(Bad, x);    // ok, returns offset of Bad::x
```

7. Proposed Wording

The proposed wording below is given relative to N4618. Inserted text is marked like this, removed text is marked like ~~this~~.

7.1. Core Wording

Modify [intro.object]/7:

Unless it is a bit-field (9.2.4), a most derived object shall have a nonzero size and shall occupy one or more bytes of storage. Base class subobjects may have zero size. An object of ~~trivially copyable or standard-layout~~stable-layout type (3.9) shall occupy contiguous bytes of storage.

Modify [basic.types]/9:

Arithmetic types (3.9.1), enumeration types, pointer types, pointer to member types (3.9.2), `std::nullptr_t`, and cv-qualified versions of these types (3.9.3) are collectively called *scalar types*. Scalar types, POD classes (Clause 9), arrays of such types and cv-qualified versions of these types (3.9.3) are collectively called *POD types*. Cv-unqualified scalar types, trivially copyable class types (Clause 9), arrays of such types, and cv-qualified versions of these types (3.9.3) are collectively called *trivially copyable types*. Scalar types, trivial class types (Clause 9), arrays of such types and cv-qualified versions of these types (3.9.3) are collectively called *trivial types*. Scalar types, standard-layout class types (Clause 9), arrays of such types and cv-qualified versions of these types (3.9.3) are collectively called *standard-layout types*. Scalar types, stable-layout class types (Clause 9), arrays of such types and cv-qualified versions of these types (3.9.3) are collectively called *stable-layout types*.

Add a new paragraph after [class]/7:

A *stable-layout class* is a class that:

- has no virtual base classes (10.1),
- has no virtual functions (10.3),
- has no non-static data members of types other than a stable-layout type (3.9) or a reference (3.9.2), and
- has no non-stable-layout base classes.

Given an object x_i of a stable-layout class X , for each non-static non-reference data member m_j of X , offset in bytes from the address of x_i to the address of m_j within x_i shall be constant and equal to the corresponding offset in any other object of class X .

Modify [class]/8:

A *standard-layout struct* is a standard-layout class defined with the *class-key* `struct` or the *class-key* `class`. A *standard-layout union* is a standard-layout class defined with the *class-key* `union`. A *stable-layout struct* is a stable-layout class defined with the *class-key* `struct` or the *class-key* `class`. A *stable-layout union* is a stable-layout class defined with the *class-key* `union`.

7.2. Library Wording

Modify [support.types.layout]/1:

The macro `offsetof(type, member-designator)` has the same semantics as the corresponding macro in the C standard library header `<stddef.h>`, but accepts a restricted set of *type* and *member-designator* arguments in this International Standard. Use of the `offsetof` macro with a *type* other than a ~~standard-layout~~stable-layout class (Clause 9) is conditionally-supported. The expression `offsetof(type, member-designator)` is never type-dependent (14.6.2.2) and it is value-dependent (14.6.2.3) if and only if *type* is dependent. The result of applying the `offsetof` macro to a static data member or a function member is undefined. If *member-designator* accesses or identifies a reference data member, the result of the `offsetof` macro is undefined. No operation invoked by the `offsetof` macro shall throw an exception and `noexcept(offsetof(type, member-designator))` shall be true.

[Example:

```
struct A { int n; };
struct B { A a; };
struct V : public virtual A { int x; };
struct R
{
    A a;
    B b[10];
    A& r;
};
struct Q : R
{
```

```

    V v;
};

void f() {
    offsetof(R, a.n); // ok
    offsetof(R, b); // ok, R::b is an array of B, which is a stable-layout class
    offsetof(R, b[5].a.n); // ok
    offsetof(R, r); // undefined behavior, member-designator identifies R::r, which is a reference
    offsetof(R, r.n); // undefined behavior, member-designator accesses R::r, which is a reference
    offsetof(V, x); // conditionally-supported, V is not a stable-layout class because of virtual inheritance
    offsetof(Q, a); // conditionally-supported, Q is not a stable-layout class because of Q::v
    offsetof(Q, v); // the same
    offsetof(Q, v.n); // the same
}

```

— *end example*

Modify [meta.type.synop]/1. After the `is_standard_layout` type trait declaration, add the new line:

```
template <class T> struct is_stable_layout;
```

In the same section, after the `is_standard_layout_v` variable template declaration, add the new declaration:

```
template <class T> constexpr bool is_stable_layout_v
= is_stable_layout<T>::value;
```

Modify [meta.unary.prop]/4, Table 42. Add a new row after `is_standard_layout` with the following contents (table header repeated for convenience):

Template	Condition	Precondition
<code>template <class T> struct is_stable_layout;</code>	<code>T is a stable-layout type (3.9)</code>	<code>remove_all_extents_t<T> shall be a complete type or cv void.</code>

8. Acknowledgements

- Thanks to all participants in the discussion of the proposal at the [std-discussion] and [std-proposals] mailing lists, and in particular to "Myriachan" <myriachan at gmail dot com> for the suggested example showing that trivially-copyable types should be supported by `offsetof`.
- Thanks to Walter E. Brown for the help with preparing and presenting the proposal.

9. References

- N4618 Working Draft, Standard for Programming Language C++ (<http://open-std.org/JTC1/SC22/WG21/docs/papers/2016/n4618.pdf>)
- Itanium C++ ABI (<https://mentorembdedded.github.io/cxx-abi/abi.html>)